Sardar Patel Institute of Technology - Department of Computer Engineering
Cryptography and Network Security Lab

Experiment 8: Evaluation of Classical and Post-Quantum Digital Signature Algorithms using Docker-Based Client-Server Communication

Tables and Graphs

Name: Anusha Dalal UID: 2024300032 Date: 16 October 2026

Table 1: Execution environment

CPU model	Cores / threads	RAM	Host OS	Docker version	cryptography version	liboqs version
Intel(R) Xeon(R) Processor @ 2.80GHz	4 / 4	15 GiB	Ubuntu 24.04.5 LTS (kernel 6.18.44-fc-v139)	29.8.2	50.0.2	0.16.0 (liboqs-python 0.16.0.1)

Host OS: Ubuntu 24.04.5 LTS. Container OS (image sig-lab:1.0, built from the Step C3 Dockerfile, FROM ubuntu:24.04): Ubuntu 24.04.5 LTS, Python 3.12.3. Container IPs on sig-net: sender 172.18.0.2, receiver 172.18.0.3.

Table 2: Key, signature and transmission sizes (bytes)

Algorithm	Public key	Private key	Signature	Document	Transmitted (doc + sig + public key)
ECDSA	91	138	71	59	221
RSA-PSS	422	1792	384	59	865
ML-DSA	1312	2560	2420	59	3791
SLH-DSA	32	64	7856	59	7947

Sizes of the files actually written and transferred, measured with stat -c %s. Key formats as prescribed in Step D3: ECDSA and RSA-PSS private keys PKCS#8 DER, public keys SubjectPublicKeyInfo DER; ML-DSA and SLH-DSA raw keys from liboqs (these match the FIPS 204/205 sizes 1312/2560/2420 and 32/64/7856 B).

Table 3: Verification results

Test case	ECDSA	RSA-PSS	ML-DSA	SLH-DSA	Expected
1. Original document + original signature	PASS	PASS	PASS	PASS	PASS
2. One character of document changed	FAIL	FAIL	FAIL	FAIL	FAIL
3. One bit of signature changed	FAIL	FAIL	FAIL	FAIL	FAIL
4. Correct document + signature, wrong public key	FAIL	FAIL	FAIL	FAIL	FAIL

Document	Text (tamper index = UID mod 59 = 2: 'a' (0x61) changed to '`' (0x60))
Original	Transfer Rs.3032 to Account 0032 / Issued by UID: 2024300032
Modified	Tr`nsfer Rs.3032 to Account 0032 / Issued by UID: 2024300032

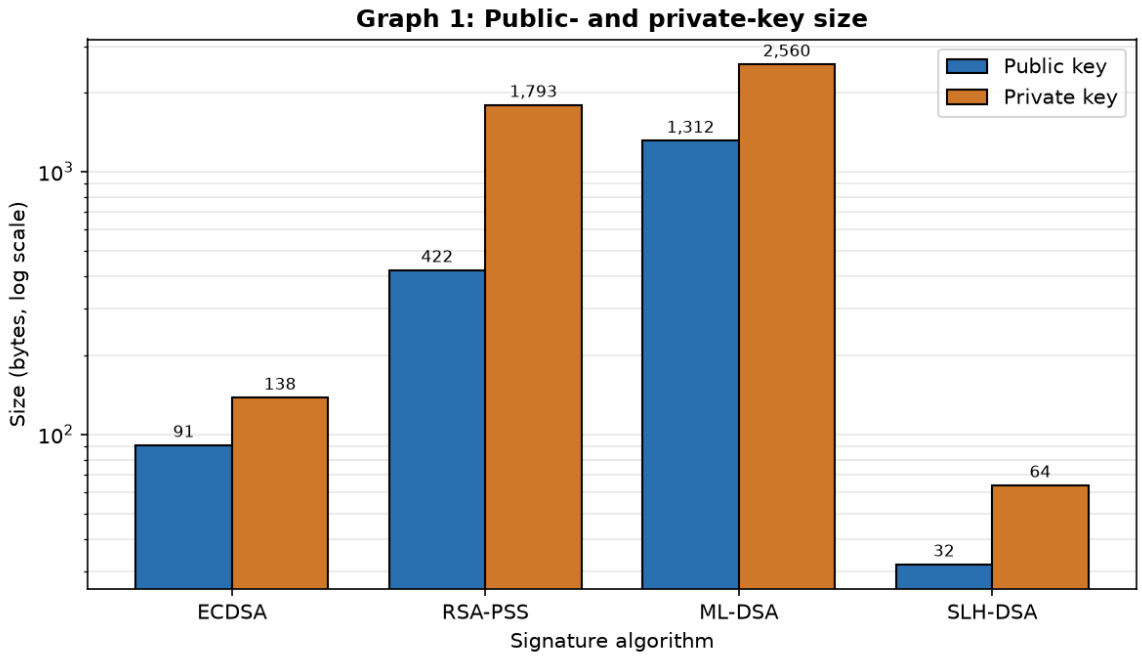
Test 3 flipped bit 0 of the signature byte at index UID mod length: ECDSA byte 4 (0x00 -> 0x01), RSA-PSS byte 256 (0x9b -> 0x9a), ML-DSA byte 1492 (0x09 -> 0x08), SLH-DSA byte 5232 (0x53 -> 0x52). Test 4: the receiver generated its own key pairs; for ECDSA an independent key from the secure random generator was used (keygen.py ecdsa --independent), because a UID-derived key would be identical to the sender's key. cmp confirmed all four receiver public keys differ from the sender's.

Table 4: Timing results (milliseconds, mean ± standard deviation)

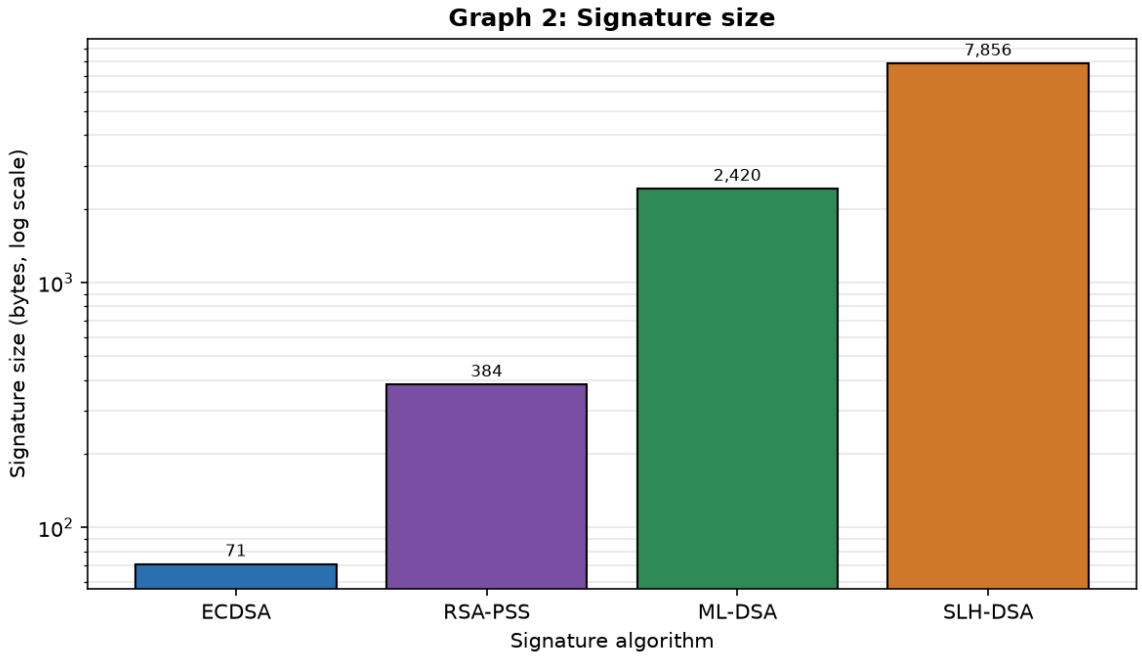
Algorithm	Key generation	Signing	Verification	Sign / Verify ratio
ECDSA	0.0765 ± 0.0187	0.0558 ± 0.0225	0.1291 ± 0.0261	0.43
RSA-PSS	215.7596 ± 172.9181	2.2231 ± 0.6104	0.0589 ± 0.0108	37.74
ML-DSA	0.0454 ± 0.0099	0.1041 ± 0.0572	0.0357 ± 0.0067	2.92
SLH-DSA	103.7940 ± 13.2166	839.7676 ± 118.5879	0.6646 ± 0.0175	1263.57

Sender container, median of three bench.py runs (results.csv = results_run2.csv); 1024-byte UID-seeded message; 5 warm-up calls, then key generation x20, signing x100, verification x100, each call timed with time.perf_counter_ns().

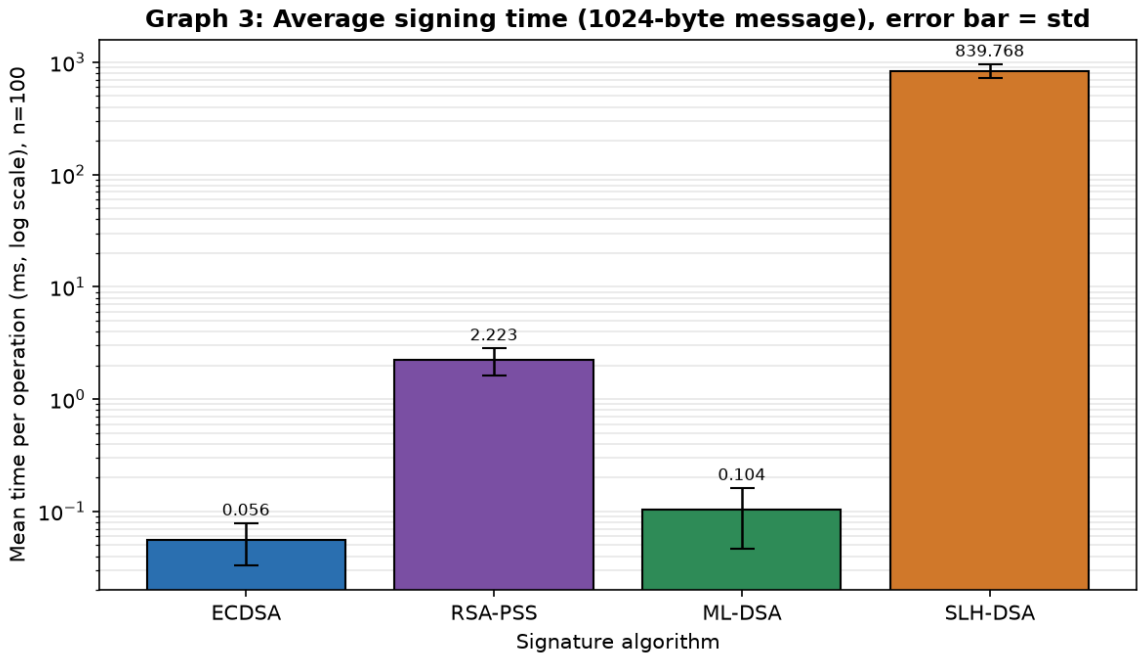
Graphs (Section 15)



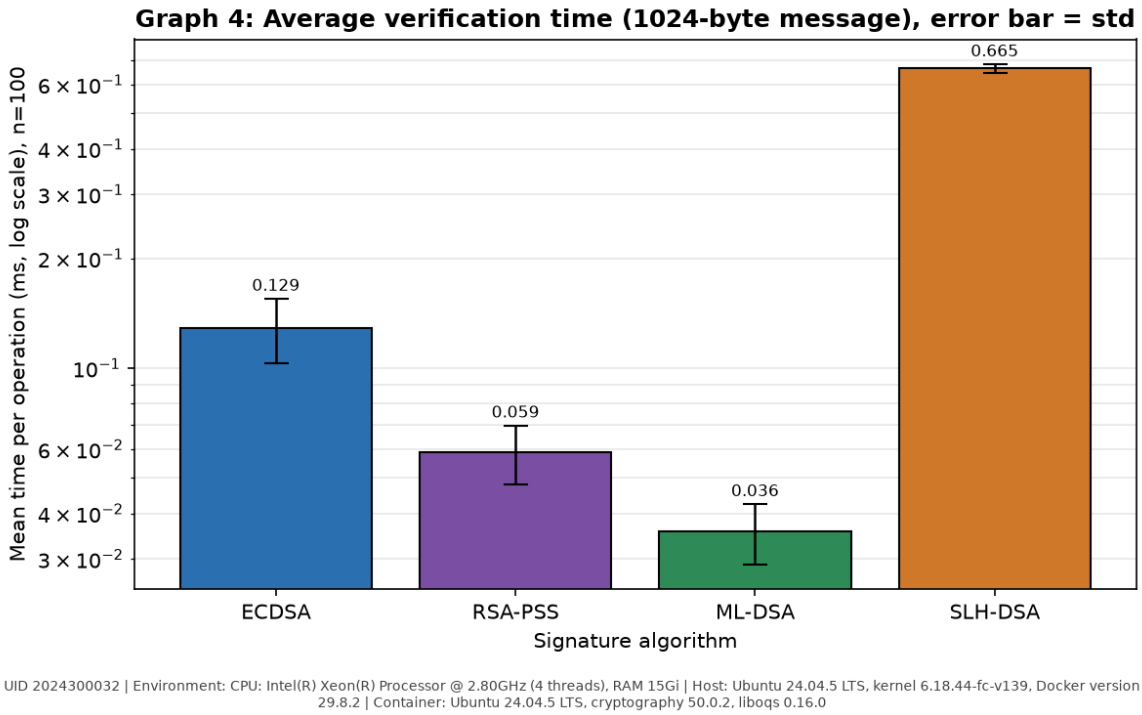
Graph 1: Public-key and private-key size (bytes, log scale). Environment: CPU: Intel(R) Xeon(R) Processor @ 2.80GHz (4 threads), RAM 15Gi; Host: Ubuntu 24.04.5 LTS, kernel 6.18.44-fc-v139, Docker version 29.8.2; Container: Ubuntu 24.04.5 LTS, cryptography 50.0.2, liboqs 0.16.0. UID 2024300032.



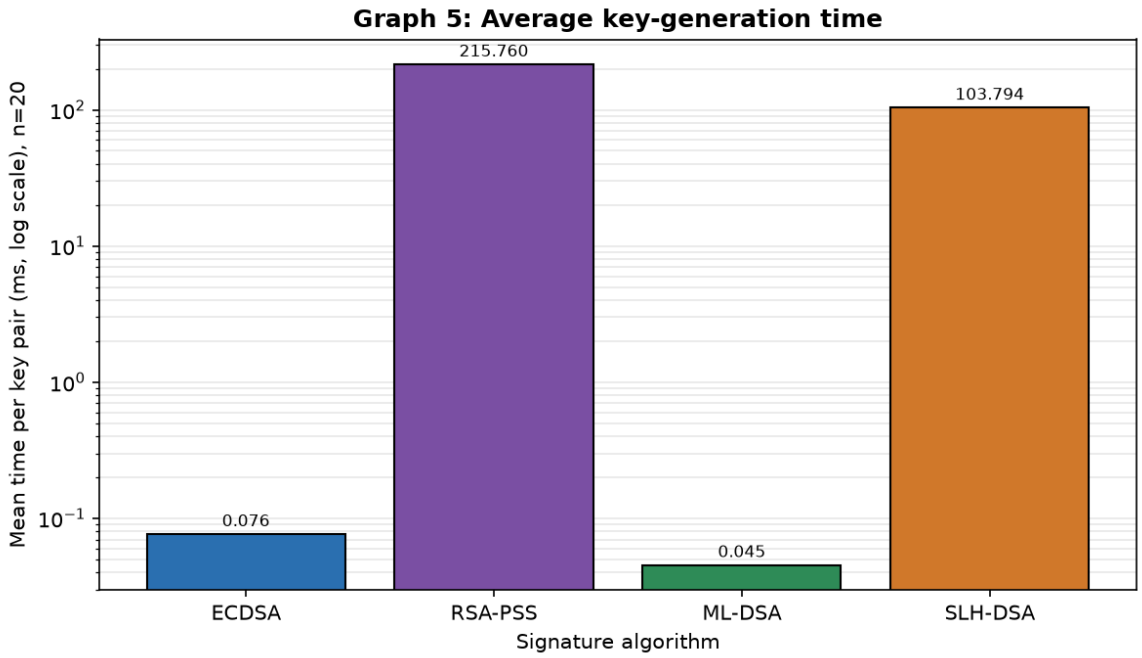
Graph 2: Signature size (bytes, log scale). Environment: CPU: Intel(R) Xeon(R) Processor @ 2.80GHz (4 threads), RAM 15Gi; Host: Ubuntu 24.04.5 LTS, kernel 6.18.44-fc-v139, Docker version 29.8.2; Container: Ubuntu 24.04.5 LTS, cryptography 50.0.2, liboqs 0.16.0. UID 2024300032.



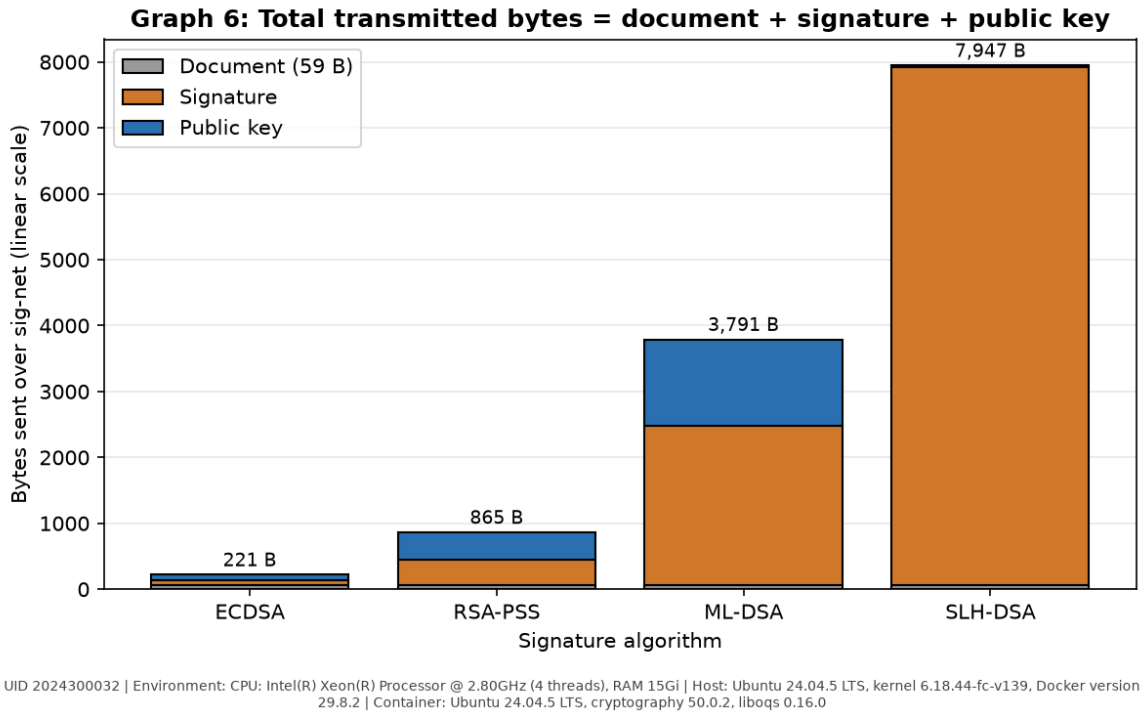
Graph 3: Average signing time (ms, log scale); error bar = standard deviation, n = 100. Environment: CPU: Intel(R) Xeon(R) Processor @ 2.80GHz (4 threads), RAM 15Gi; Host: Ubuntu 24.04.5 LTS, kernel 6.18.44-fc-v139, Docker version 29.8.2; Container: Ubuntu 24.04.5 LTS, cryptography 50.0.2, liboqs 0.16.0. UID 2024300032.



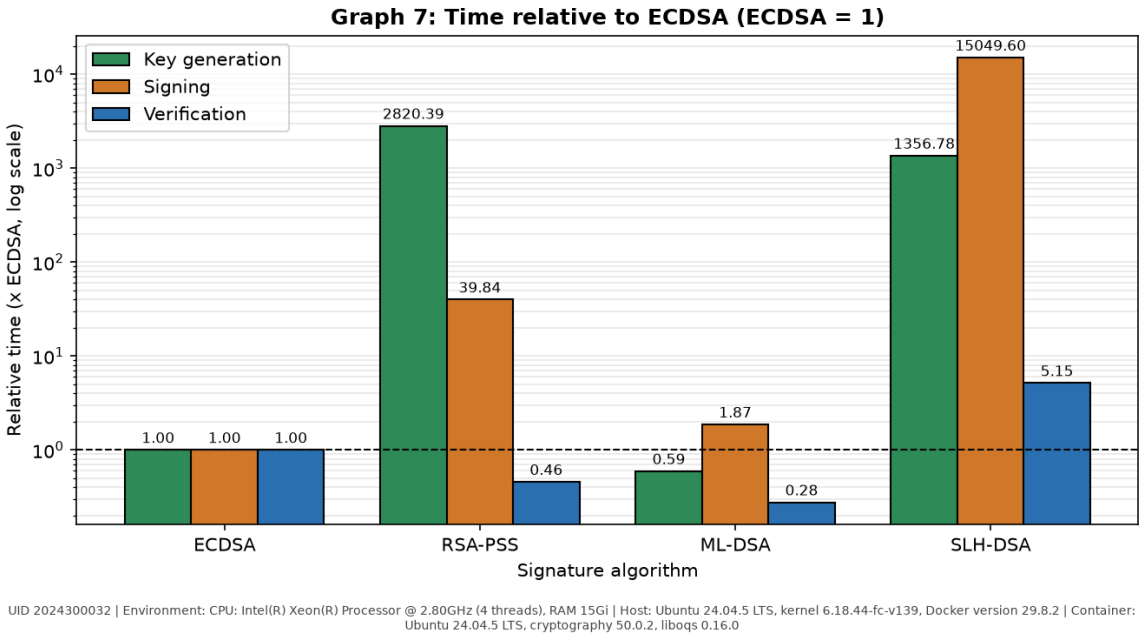
Graph 4: Average verification time (ms, log scale); error bar = standard deviation, n = 100. Environment: CPU: Intel(R) Xeon(R) Processor @ 2.80GHz (4 threads), RAM 15Gi; Host: Ubuntu 24.04.5 LTS, kernel 6.18.44-fc-v139, Docker version 29.8.2; Container: Ubuntu 24.04.5 LTS, cryptography 50.0.2, liboqs 0.16.0. UID 2024300032.



Graph 5: Average key-generation time (ms, log scale), n = 20. Environment: CPU: Intel(R) Xeon(R) Processor @ 2.80GHz (4 threads), RAM 15Gi; Host: Ubuntu 24.04.5 LTS, kernel 6.18.44-fc-v139, Docker version 29.8.2; Container: Ubuntu 24.04.5 LTS, cryptography 50.0.2, liboqs 0.16.0. UID 2024300032.



Graph 6: Total transmitted bytes = document + signature + public key. Environment: CPU: Intel(R) Xeon(R) Processor @ 2.80GHz (4 threads), RAM 15Gi; Host: Ubuntu 24.04.5 LTS, kernel 6.18.44-fc-v139, Docker version 29.8.2; Container: Ubuntu 24.04.5 LTS, cryptography 50.0.2, liboqs 0.16.0. UID 2024300032.



Graph 7 (extra): Sign, verify and key-generation time relative to ECDSA (ECDSA = 1, log scale). Environment: CPU: Intel(R) Xeon(R) Processor @ 2.80GHz (4 threads), RAM 15Gi; Host: Ubuntu 24.04.5 LTS, kernel 6.18.44-fc-v139, Docker version 29.8.2; Container: Ubuntu 24.04.5 LTS, cryptography 50.0.2, liboqs 0.16.0. UID 2024300032.

Table 5: Relative comparison with ECDSA as the baseline (measured value / ECDSA value)

Parameter	ECDSA	RSA-PSS	ML-DSA	SLH-DSA
Public key size	1.0	4.64	14.42	0.35
Signature size	1.0	5.41	34.08	110.65
Signing time	1.0	39.84	1.87	15049.60
Verification time	1.0	0.46	0.28	5.15
Key generation time	1.0	2820.39	0.59	1356.78
Total transmitted bytes	1.0	3.91	17.15	35.96

Table 6: Property comparison

Property	ECDSA	RSA-PSS	ML-DSA	SLH-DSA
Standard	FIPS 186-5	RFC 8017, FIPS 186-5	FIPS 204 (2024)	FIPS 205 (2024)
Security parameter / category	P-256 (secp256r1), ~128-bit	RSA-3072, ~128-bit	ML-DSA-44, NIST category 2	SLH-DSA-SHA2-128s, NIST category 1
Mathematical basis	Elliptic-curve discrete logarithm	Integer factorisation	Module lattices (Module-LWE / SIS)	Hash functions only (hypertree of Merkle / FORS trees)
Resistant to Shor's algorithm?	No	No	Yes	Yes
Public key size (small / medium / large)	Small (91 B)	Medium (422 B)	Large (1312 B)	Small (32 B)
Signature size	Small (71 B)	Medium (384 B)	Large (2420 B)	Very large (7856 B)
Fastest operation	Signing (0.056 ms)	Verification (0.059 ms)	Verification (0.036 ms)	Verification (0.665 ms)
Slowest operation	Verification (0.129 ms)	Key generation (215.8 ms)	Signing (0.104 ms)	Signing (839.8 ms)
Maturity / deployment	Very mature; TLS, SSH, code signing, smart cards	Very mature (decades); PKI, TLS, document signing	New (2024); being rolled out, often in hybrid mode	New (2024), conservative SPHINCS+ design; firmware, root keys
Main advantage	Smallest keys and signatures, fast signing	Very fast verification, universal support	Quantum-safe and fast (fastest verification measured)	Quantum-safe, relies only on hash functions; 32 B public key
Main disadvantage	Broken by Shor; needs secure nonces	Broken by Shor; slow key generation, larger keys	Large public key and signature	Very slow signing, very large signature

Table 7: Which algorithm would you choose?

Scenario	Chosen algorithm	Justification
Signing a firmware image for a device that stays in the field for 15 years	SLH-DSA	Must outlive quantum computers; hash-only security is the most conservative, signing is rare and verification was cheap in our tests.
Authenticating a short sensor message on a very low-bandwidth link	ECDSA	Smallest overhead (71 B signature, 91 B key) and the data is short-lived.
Signing a TLS server certificate in 2026 on a general web server	ECDSA (P-256)	Universally supported, tiny handshake; certificates live < 1 year, with a planned move to hybrid ECDSA + ML-DSA.
Root key of a certificate authority that must remain trusted for 30 years	SLH-DSA	Rarely signs, so slow signing and large signatures are acceptable; security rests only on hash functions.
High-volume server that verifies millions of signatures per second	ML-DSA	Fastest verification in our measurements (0.036 ms per signature on this setup) and quantum-safe.

Timings in Tables 4-7 are those measured in this experiment's containers on the machine in Table 1; absolute values depend on the hardware and library build, the relative ordering is what the choices rely on.